📖 DOCUMENTACIÓN OFICIAL DEL PROYECTO SGDM — ASCEND

Sistema de Gestión Deportiva Modular — Plataforma ASCEND

> **Versión:** 2.0 — Segunda Entrega
> **Fecha:** 14 de septiembre de 2026
> **Equipo:** Grupo ASCEND — 3° BT Tecnologías de la Información
> **Institución:** Instituto Tecnológico Superior Arias-Balparda

FASE 1 — INICIO DEL PROYECTO (Días 1 a 15)

1.1 Acta de Constitución del Proyecto (Project Charter)

Información General

El presente documento constituye el acta formal de inicio para el **Sistema de Gestión Deportiva Modular (Plataforma ASCEND)**, un proyecto integrador desarrollado por el equipo ASCEND del 3° BT Tecnologías de la Información. El proyecto cuenta con el patrocinio académico de la Coordinación de ITS Arias-Balparda y se ejecutará a lo largo de 180 días calendario (con un tiempo de desarrollo efectivo estimado en 120 días), iniciando el 14 de mayo de 2026 y concluyendo con las defensas finales a finales de noviembre de 2026.

Justificación del Proyecto

El ecosistema deportivo y recreativo actual adolece de una gran fragmentación en sus herramientas de gestión. Los organizadores de competencias se ven continuamente obligados a depender de planillas de cálculo manuales, aplicaciones móviles genéricas dispersas y formatos de registro no estandarizados. Esta situación genera un alto margen de error humano en la administración de calendarios, resultados y posiciones. ASCEND surge como una respuesta directa a esta problemática, proponiendo una solución web modular, unificada y altamente extensible, capaz de centralizar la administración de torneos deportivos, mentales y electrónicos bajo un único paraguas tecnológico.

Objetivos SMART

Nuestros objetivos se definen de la siguiente manera:

- **Específico (S):** Construir una plataforma web utilizando arquitectura MVC sobre PHP 8.x y MySQL. El sistema gestionará competencias en tres formatos consolidados (Liga, Eliminación Directa y Sistema Suizo) abarcando 9 disciplinas distintas.
- **Medible (M):** Cumplimentar a cabalidad el 100% de los 74 requisitos funcionales (RF-001 a RF-074) y los 29 requisitos no funcionales (RNF01 a RNF29) relevados.
- **Alcanzable (A):** Ejecutar el desarrollo con un equipo cohesionado de 4 especialistas, utilizando herramientas open-source (XAMPP, GitHub) y basándose en una sólida documentación técnica.

- **Relevante (R):** Integrar de forma orgánica y demostrable las competencias adquiridas en las 5 asignaturas troncales del currículo (Programación Fullstack, Ingeniería de Software, Administración de Sistemas Operativos, Ciberseguridad y Tutoría UTULAB).
- **Temporal (T):** Respetar de forma irrestricta los hitos de entrega pautados para el 27 de julio, 14 de septiembre y 9 de noviembre de 2026.

Restricciones y Asunciones

El proyecto asume restricciones tecnológicas claras, como la obligatoriedad de utilizar PHP 8.x nativo (sin frameworks comerciales como Laravel o Symfony) y un presupuesto de infraestructura nulo (USD 0), apoyándose enteramente en tecnologías open-source y despliegue local. Partimos de la premisa de que todos los integrantes del equipo disponen del entorno de desarrollo adecuado (XAMPP/Docker) y que la comunicación fluirá a través de canales oficiales establecidos, como Discord para la inmediatez y GitHub para el control de versiones.

1.2 Resumen Ejecutivo del Proyecto

ASCEND se perfila como una plataforma web integral concebida para la gestión de torneos multidisciplinarios. Su arquitectura permite a diferentes actores (administradores, organizadores y jugadores) coexistir e interactuar de forma segura en un ecosistema competitivo organizado.

El sistema da soporte nativo a 9 disciplinas variadas que abarcan deportes físicos (Fútbol, Básquetbol, Tenis), deportes electrónicos o e-sports (League of Legends, Counter-Strike 1.6, EA Sports FC 26) y deportes mentales o de mesa (Truco, Catán, Ajedrez). Para dotar de dinamismo a estas disciplinas, ASCEND incorpora potentes motores de cálculo para tres formatos de competencia: Liga (todos contra todos), Eliminación Directa (llaves o brackets) y Sistema Suizo (emparejamiento adaptativo por rendimiento).

A nivel estructural, el núcleo del software descansa sobre un estricto patrón **Modelo-Vista-Controlador (MVC)**, garantizando la separación de preocupaciones. La persistencia de datos se apoya en una base de datos MySQL rigurosamente normalizada hasta la Tercera Forma Normal (3FN), implementando un modelo de herencia de tablas (Class Table Inheritance) que eleva la robustez y escalabilidad de la información. El acceso a las funcionalidades está regido por un Sistema de Control de Acceso Basado en Roles (RBAC).

1.3 Caso de Negocio (Business Case)

La gestión de torneos amateurs e institucionales presenta fricciones considerables. El uso de herramientas desconectadas (Excel, grupos de WhatsApp, anotadores) deriva en errores de emparejamiento, resultados duplicados y pérdida del registro histórico de los competidores. Paralelamente, las plataformas profesionales existentes imponen barreras de entrada, ya sea por sus costos de licenciamiento, su falta de localización al español, o su excesiva complejidad para el usuario promedio.

Retorno Esperado: ASCEND democratiza el acceso a la gestión deportiva profesional. Su adopción se traduce en una reducción drástica (estimada en un 80%) del tiempo administrativo requerido para orquestar un torneo. El motor algorítmico elimina por completo el error humano en los sorteos y recálculo de tablas de posiciones. Para el Instituto y el equipo desarrollador, representa un retorno académico invaluable al plasmar en código real los conocimientos teóricos del Tramo 8.

1.4 Agenda de la Reunión de Inicio (Kick-off Meeting)

Durante el arranque del proyecto, el equipo se consolidó para analizar minuciosamente la letra del proyecto (`Proyecto.md`). Se debatieron los roles naturales de cada integrante, alineando las fortalezas individuales con las necesidades arquitectónicas del sistema.

Se oficializó el nombre **ASCEND** (A Sports Competition Engine for Networked Disciplines) como insignia del producto, y se definió una estrategia de ciclo de vida híbrido, combinando la planificación exhaustiva del modelo en Cascada con la flexibilidad iterativa y de integración de Scrum. El repositorio fue configurado para proteger la rama `main`, canalizando los esfuerzos diarios mediante ramas de característica (`feat/*`).

1.5 Registro de Interesados (Stakeholder Register) y Matriz de Poder

Nuestros interesados abarcan desde el tribunal académico y la coordinación docente (quienes poseen un alto poder de evaluación y un alto interés en la calidad del resultado), hasta el propio equipo desarrollador, que funge como el motor de ejecución. Del lado del producto, identificamos al Administrador del sistema, a los Organizadores (usuarios críticos que validan el valor de la plataforma) y a los Jugadores. Finalmente, el público general conforma una base de usuarios que consume los datos de forma pasiva.

1.6 Documento de Viabilidad y Alcance

Análisis de Viabilidad

Técnicamente, el stack conformado por PHP 8.x, Apache y MySQL (vía XAMPP) es ampliamente dominado por el equipo y cumple a cabalidad con las exigencias del programa de Programación Fullstack. Operativamente, las bases teóricas de Ingeniería de Software proveen el andamiaje metodológico para no desviar el rumbo, mientras que económicamente el proyecto es sostenible debido al aprovechamiento de recursos de hardware propios y software libre.

Fronteras del Alcance

El sistema entregará de forma **end to end** la gestión de usuarios, torneos, generación de fixtures, carga de resultados y publicación de estadísticas. Quedan excluidas explícitamente —de acuerdo a las directrices originales— integraciones con pasarelas de pago para inscripciones, desarrollo de inteligencia artificial para arbitrajes, sincronización con APIs de redes sociales, y cualquier tipo de gestión de acceso físico a las instalaciones de los eventos.

1.7 Registro de Requisitos Oficiales

El levantamiento de requisitos se gestó mediante una lectura crítica de la documentación base, entrevistas y cuestionarios de campo, logrando aislar 74 Requisitos Funcionales y 29 Requisitos No Funcionales. Todo este exhaustivo trabajo se encuentra formalmente documentado en nuestros archivos maestros: `RequisitosFuncionales.md` y `RequerimientosNoFuncionales.md`, los cuales rigen el alcance exacto del desarrollo.

(Para más detalle, referirse a los documentos alojados en la unidad compartida y en la carpeta local `Documentos_Juan`).

FASE 2 — PLANIFICACIÓN DEL PROYECTO (Días 16 a 45)

2.1 Enfoque Metodológico: Modelo Híbrido

Para sortear la rigidez de una metodología puramente predictiva y el potencial caos de una metodología ágil sin marco, ASCEND se concibió bajo un modelo híbrido.

- **La Planificación y el Diseño** (Base de datos, UML, Casos de Uso, Documentación) se ejecutaron con rigor de *Waterfall*.
- **La Construcción y Codificación** se desarrollan mediante iteraciones y sprints quincenales estilo *Scrum*, permitiendo una rápida adaptación a problemas arquitectónicos como los que enfrentamos durante la creación de los motores de competencia.

2.2 Estructura de Desglose del Trabajo (EDT)

El trabajo se ramificó lógicamente en seis grandes paquetes:

1. **Arquitectura y Diseño:** MER, DER, UML, y definición del MVC.
2. **Backend PHP:** Módulos de Autenticación, Administración, Organización y Jugador, más el corazón del sistema: los Algoritmos de Competencia.
3. **Frontend:** Interfaces gráficas, maquetación "Mobile First" y experiencia de usuario.
4. **Base de Datos:** Construcción física del esquema SQL, DCL y scripts de carga.
5. **Infraestructura:** Servicios del sistema operativo, contenedores Docker y políticas de resiliencia.
6. **Documentación Oficial:** Generación de manuales y carpetas académicas por materia.

2.3 Plan de Gestión de Recursos (El Equipo ASCEND)

El éxito de ASCEND recae en la sinergia y la especialización de sus cuatro desarrolladores. La asignación de roles se diseñó para maximizar las destrezas de cada integrante y cubrir todas las aristas técnicas del sistema:

1. **Juan (Líder del Proyecto, Arquitecto Backend y DBA):**
    - *Responsabilidades:* Encargado de la arquitectura global del patrón MVC, diseño, normalización y mantenimiento de la base de datos (DBA) en 3FN, y construcción del módulo Administrador. Adicionalmente, consolida la documentación técnica e institucional, asegurando el cumplimiento metodológico.
2. **Michael (Desarrollador Backend - Módulo Organizador):**
    - *Responsabilidades:* Responsable de la lógica de negocio y controladores orientados a los organizadores de torneos. Su labor incluye la programación para la creación, configuración y gestión del ciclo de vida de los campeonatos, así como la carga de resultados.
3. **Valentina (Desarrolladora Backend - Módulo Jugador):**
    - *Responsabilidades:* Encargada de dar vida a la experiencia del participante. Desarrolla la gestión de perfiles de jugadores, la lógica de creación de equipos, el intrincado sistema de invitaciones,

validaciones de los referentes y el registro a las competencias.

4. **Mauricio (Desarrollador Frontend, Vistas Públicas y QA):**
    - *Responsabilidades:* A cargo de la interfaz gráfica y la maquetación. Es el principal responsable del módulo de vista pública (consultas de rankings y torneos sin necesidad de login) y ejecuta el rol crítico de Quality Assurance (QA) probando la integración del código y garantizando una experiencia de usuario (UX) pulida.

2.4 Documentación Complementaria por Materia

Cabe destacar que, si bien la presente es la *Documentación de Programación*, el proyecto se nutre de una extensa biblioteca de documentos académicos generados para las distintas asignaturas involucradas. Estos artefactos **existen y están alojados formalmente** en nuestra estructura de trabajo (referenciados desde `documentacion_drive/Proyecto 2026-20260909T173932Z-1-001/Proyecto 2026/`):

- **Ingeniería de Software:** Allí radican nuestro cronograma oficial (`1eraEntrega (1).gan`), el **Estudio de Factibilidad** pormenorizado, y las matrices preliminares de requerimientos (`Documento Ingenieria.docx`, `RequisitosFuncionalesAscend.odt`).
- **Tutoría UTULAB (Actas de Equipo):** Contamos con un registro intachable del desarrollo organizativo del equipo. En la carpeta `Actas de equipo` se custodian más de 12 documentos formales (desde la `Primera Acta` hasta la `Duodécima Reunión`), respaldando nuestras retrospectivas, debates de arquitectura y toma de decisiones (FODA, Costo-Beneficio).
- **Sistemas Operativos (SO):** En su respectiva carpeta se alojan los scripts de infraestructura (`ScriptUsarioGrupo.sh`, `gestion_logs.sh`) y el documento maestro `Sistema Operativo.docx` que aborda políticas de gestión, contingencias y monitoreo de servidores.

Esta segmentación asegura que cada área evaluadora encuentre sus insumos en el formato y nivel de profundidad esperado.

2.5 Gestión de Riesgos, Comunicaciones y Calidad

El proyecto identificó riesgos latentes, principalmente enfocados en los inminentes conflictos de integración de código (merges en Git) al tener a tres programadores interactuando fuertemente sobre las clases del Backend. Como mitigación, Juan centralizó la revisión arquitectónica de los merges más complejos (como el modelo `Torneo.php` o `Sesion.php`).

La comunicación se estructuró sobre Discord para la inmediatez, dejando GitHub como el registro de la verdad para el código.

A nivel de **Calidad**, el equipo juró lealtad al estándar **PSR-12** para PHP, dotando al código de una legibilidad impecable. Se instauró una regla de oro inquebrantable en el frontend: **"Cero estilos en línea (inline styles)"**, forzando la separación entre la estructura (HTML) y la presentación (CSS). En materia de seguridad, se adoptaron lineamientos de **OWASP**, utilizando contraseñas cifradas con `bcrypt` y anulando riesgos de inyección SQL mediante sentencias preparadas (PDO).

FASE 3 — EJECUCIÓN DEL PROYECTO (Días 46 a 90)

3.1 Progreso y Sprints

Durante la fase de construcción, el trabajo se fragmentó en módulos atacados de forma paralela y secuencial:

- El diseño de la base de datos (Juan) sirvió como cimiento temprano.
- La autenticación sentó la base de seguridad.
- Valentina desplegó el registro y creación de equipos para los jugadores, una tarea compleja que exigió el manejo transaccional de invitaciones.
- Michael abordó el núcleo de la organización, permitiendo que un torneo nazca y se gestione íntegramente.
- El panel de administración fue orquestado por Juan para mantener la gobernanza global del sistema.

3.2 Decisiones Técnicas Relevantes (Arquitectura y Código)

Durante la ejecución, tomamos decisiones que marcaron la identidad del sistema:

- **Herencia en Base de Datos (Class Table Inheritance):** Para evitar una tabla de `usuarios` colapsada por columnas nulas o irrelevantes, modelamos `usuarios` como la tabla padre, de la cual heredan `perfiles_jugadores` y `perfiles_organizadores` compartiendo la misma clave primaria. Esto garantiza la normalización total y simplifica las consultas.
- **Controlador Frontal y Singleton para PDO:** Implementamos `Conexion.php` mediante el patrón Singleton, asegurando que la conexión a MySQL instancie un solo objeto a lo largo de toda la petición HTTP, economizando recursos.
- **Session Management Desacoplado:** Centralizamos el control de sesiones y los chequeos de rol en la clase `Sesion.php`. Esta pieza maestra (RBAC) protege cualquier ruta y efectúa la regeneración de la ID de sesión, previniendo ataques de fijación (Session Fixation).
- **Motor de Competencia Unificado:** En lugar de crear un modelo por cada deporte, concentramos la lógica algorítmica de los fixtures en `Torneo.php`, permitiendo que el formato (Liga, Suizo, Eliminación) defina el comportamiento.

FASE 4 — MONITOREO Y CONTROL (Días 91 a 110)

4.1 Informe de Estado Actual (Día 120 - Segunda Entrega)

Nos encontramos ante un producto sustancialmente maduro.

- La **Base de Datos** se encuentra al 100% de su estructura conceptual, lógica y física.
- La **Autenticación (RBAC)** opera fluidamente.
- El **Panel del Organizador** (Michael) y el **Panel del Jugador** (Valentina) están plenamente entrelazados.
- El **Motor de Torneos**, el corazón del sistema, es capaz de calcular fixtures de Liga, sortear brackets de Eliminación Directa y orquestar las complejas reglas de emparejamiento por rendimiento del Sistema Suizo.
- Las **Vistas Públicas** (Mauricio) han sido maquetadas con esmero bajo una filosofía "Mobile First", aunque aún se encuentran en etapa de acoplamiento dinámico final.

Nuestros reportes de auditoría interna señalan que más del 70% del código está funcional y estabilizado, listos para recibir las integraciones finales y la orquestación mediante contenedores Docker para la tercera entrega.

4.2 Matriz de Trazabilidad: Del Requisito al Código

Una de las grandes fortalezas del equipo fue asegurar que ningún fragmento de código fuese arbitrario. Todo parte de un Requisito Funcional (RF):

- Cuando el RF-008 exige la "*Creación de equipos*", Valentina implementó el modelo `Equipo.php` y el `EquipoControlador.php`, mapeados contra las tablas `equipos` y `equipo_miembros`.
- Cuando el RF-017 demanda la "*Creación de torneos*", Michael interactúa directamente con `Torneo.php`, alimentando las tablas `torneos` y `torneo_config`.
- Cuando el RF-074 requiere "*Registro de actividad*", Juan desplegó los logs de auditoría respaldados por el DCL (Data Control Language) y el helper de `Auditorias.php`.

FASE 5 — CIERRE Y PROYECCIÓN HACIA LA ENTREGA FINAL

5.1 Lecciones Aprendidas

El camino de ASCEND ha sido una constante curva de aprendizaje:

1. **La comunicación en Git es vital:** Subestimamos inicialmente el costo de resolver conflictos de merge en archivos de altísima concurrencia (como `Torneo.php`). La modularización nos obligó a comunicarnos antes de hacer `push`.
2. **Los algoritmos teóricos son complejos en la práctica:** Trasladar las reglas de un Sistema Suizo al código PHP implicó profundas refactorizaciones para evitar que dos jugadores se enfrentasen repetidamente.
3. **El valor de la normalización temprana:** Invertir semanas en discutir la Tercera Forma Normal (3FN) antes de tocar el código nos ahorró incontables horas de rediseño. La estructura de la base de datos absorbió el crecimiento de los requisitos sin fracturarse.

5.2 Hacia la Tercera Entrega (Punch List)

Para coronar el proyecto con éxito en noviembre, nuestros cañones apuntan a:

- Terminar de inyectar datos reales (PHP backend) en las maravillosas vistas públicas de Mauricio.
- Desplegar nuestro entorno sobre contenedores **Docker**, estandarizando la infraestructura.
- Automatizar respaldos y consolidar el monitoreo de los servicios del sistema operativo.
- Ejecutar una batería exhaustiva de testing unitario y refinar nuestra defensa ante los tribunales evaluadores.

---

6. DICCIONARIO DE REFERENCIAS Y GLOSARIO TÉCNICO

Para facilitar la lectura y comprensión de nuestra documentación técnica y arquitectónica, definimos los siguientes acrónimos y conceptos recurrentemente utilizados por el equipo ASCEND:

Término	Definición y Contexto en el Proyecto
End to End (E2E)	Hace referencia a la cobertura completa de un proceso de principio a fin, desde la interfaz gráfica del usuario (Frontend) hasta el procesamiento en el servidor y la persistencia en la Base de Datos (Backend).
OWASP	*(Open Worldwide Application Security Project)*. Es el estándar global de seguridad de software. En ASCEND nos basamos en sus normativas (OWASP Top 10) para aplicar contraseñas con Hash, prevenir inyecciones SQL (con PDO) y ataques XSS o fijación de sesiones.
PSR-12	Estándar oficial de estilo de codificación para PHP. Define reglas estrictas sobre indentación, uso de mayúsculas, espacios y organización de clases, permitiendo que el código de Juan, Michael y Valentina luzca homogéneo.
RBAC	*(Role-Based Access Control)*. Sistema de seguridad y permisos basado en los roles del usuario. En ASCEND, determina si la persona autenticada es un Administrador, Organizador o Jugador, restringiendo el acceso a los controladores.
DBA	*(Database Administrator)*. Rol técnico encargado de diseñar, optimizar y proteger la base de datos. En el equipo, Juan ejecutó las tareas de DBA diseñando la normalización 3FN y las políticas DCL (Data Control Language).
EDT / WBS	*(Estructura de Desglose del Trabajo / Work Breakdown Structure)*. Herramienta de planificación de proyectos que subdivide el alcance total del proyecto en porciones de trabajo manejables (por ejemplo: Fase Frontend, Fase Base de Datos).
MVC	*(Modelo-Vista-Controlador)*. Patrón de arquitectura de software que separa la lógica de acceso a datos (Modelo), la interfaz de usuario (Vista) y las reglas de negocio (Controlador), haciendo el código altamente mantenible.
3FN	*(Tercera Forma Normal)*. Estándar de diseño de bases de datos relacionales que garantiza que no existan redundancias ni dependencias transitivas en las tablas.
Class Table Inheritance	Patrón de modelado de bases de datos donde una jerarquía de herencia orientada a objetos (ej. Usuario -> Jugador/Organizador) se mapea a múltiples tablas con relaciones de uno a uno (1:1).
CR	*(Change Request - Solicitud de Cambio)*. Documentación formal de cualquier alteración al alcance original del software que requiere análisis de impacto en tiempos y costos.

> **Fin del Documento — Versión Oficial 2.0 (Segunda Entrega)**
> *Grupo ASCEND — 2026*